

엣지 실리콘에서 스트리밍 VLM 양자화

양자화만으론 부족하다: 융합커널 런타임으로 작고 빠르게

2i

2026-07-29

초록

4B급 스트리밍 비전-언어모델(VLM) 하나를 데이터센터 GPU가 아니라 소비자용 Apple 실리콘 Mac 한 대에 올려 실행했다. 대상은 Apache-2.0 라이선스로 공개된 microsoft/Mage-VL이며, Mage-ViT 비전 인코더와 Qwen3-4B-Instruct-2507 언어 백본을 결합한 구조다. Apple M4 Pro(통합 메모리 48GB) 위에서 bf16, INT8 weight-only 양자화, INT4 weight-only 양자화 세 가지 정밀도로 이미지 이해 추론을 실제로 돌리고 디바이스 메모리, 디코드 처리량, 첫 토큰까지 걸리는 시간, 답변 품질을 측정했다. 결과는 처음엔 절반의 성공, 절반의 반박으로 갈렸다. 양자화는 디바이스 메모리를 bf16의 9.49GB에서 INT8 5.86GB(38% 감소), INT4 4.15GB(56% 감소)로 확실히 줄였지만, 같은 양자화가 transformers + Metal(MPS) 경로에서 디코드 속도는 정반대로 움직였다. bf16 13.01 tok/s에서 INT8 2.31 tok/s, INT4 0.47 tok/s로 오히려 느려졌다. 저비트 행렬곱을 위한 융합 커널이 없는 런타임에서는 양자화가 메모리 절감만 주고 속도는 갇아야 할 빛으로 돌아온다는 뜻이었다.

목차

1 요약	3
2 배경	3
2.1 왜 엣지에서 VLM을 돌리려 하는가	3
2.2 모델 구조	3
2.3 저자 보고 수치 (참고, 우리 측정 아님)	4
3 방법	4
3.1 하드웨어와 스택	4
3.2 양자화 방법 (두 경로)	4
3.3 테스트 입력	4
3.4 측정 지표	4
4 결과	5
4.1 측정표	5
4.2 그림: 메모리와 디코드 처리량, 정밀도와 런타임 둘 다 바뀌서	5
5 해석: 런타임이 레버다	6
6 한계와 스코프	6
7 데이터와 재현	7
8 참고문헌	7

1 요약

4B급 스트리밍 비전-언어모델(VLM) 하나를 데이터센터 GPU가 아니라 소비자용 Apple 실리콘 Mac 한 대에 올려 실행했다. 대상은 Apache-2.0 라이선스로 공개된 microsoft/Mage-VL이며, Mage-ViT 비전 인코더와 Qwen3-4B-Instruct-2507 언어 백본을 결합한 구조다. Apple M4 Pro(통합 메모리 48GB) 위에서 bf16, INT8 weight-only 양자화, INT4 weight-only 양자화 세 가지 정밀도로 이미지 이해 추론을 실제로 돌리고 디바이스 메모리, 디코드 처리량, 첫 토큰까지 걸리는 시간, 답변 품질을 측정했다. 결과는 처음엔 절반의 성공, 절반의 반박으로 갈렸다. 양자화는 디바이스 메모리를 bf16의 9.49GB에서 INT8 5.86GB(38% 감소), INT4 4.15GB(56% 감소)로 확실히 줄였지만, 같은 양자화가 transformers + Metal(MPS) 경로에서 디코드 속도는 정반대로 움직였다. bf16 13.01 tok/s에서 INT8 2.31 tok/s, INT4 0.47 tok/s로 오히려 느려졌다. 저비트 행렬곱을 위한 융합 커널이 없는 런타임에서는 양자화가 메모리 절감만 주고 속도는 갇아야 할 빛으로 돌아온다는 뜻이었다.

이번 갱신에서 이 열린 질문을 실측으로 답았다. 같은 Apple M4 Pro에서, 같은 언어 백본(Qwen3-4B-Instruct-2507)을 mlx-lm의 융합 커널 INT4 경로로 돌리자 디코드 처리량이 **89.67 tok/s**, 프롬프트 처리량이 71.06 tok/s, 피크 메모리가 **2.46GB**로 나왔다. optimum-quanto의 weight-only INT4가 같은 4비트 정밀도에서 0.47 tok/s였던 것과 비교하면, 같은 비트폭에서 **약 190배 빠른 디코드를** 얻으면서 메모리도 더 작았다. 이것은 이 논문이 처음 제기했던 가설, 즉 “메모리와 속도는 서로 다른 축이고, 엣지 양자화가 실제 속도 이득까지 내려면 저비트 표현을 직접 계산하는 융합 커널 런타임이 필요하다”는 주장을 정면으로 증명하는 결정적 데이터다. 다만 정직하게 밝힐 경계가 있다. 이 89.67 tok/s는 Mage-VL 전체가 아니라 언어 백본만을 MLX로 격리해 측정한 값이다. 디코드는 언어모델 바운드 연산이라(비전 인코더는 프리필 시점에 한 번만 관여) 이 격리 측정이 전체 VLM 디코드 속도를 지배하는 요인을 정확히 짚어내지만, Mage-ViT 비전 인코더까지 포함한 Mage-VL 전체가 MLX에서 엔드투엔드로 도는 것은 아니다. 그러려면 커스텀 아키텍처 mage_vl을 mlx-vlm에 포팅하는 작업이 필요하며, 이것이 이 논문이 도출하는 구체적이고 판매 가능한 다음 패키징 작업이다. 스트리밍 비디오의 신경 코덱 경로(DCVC-RT)는 여전히 CUDA에 묶여 있어 이 논문의 스코프 밖이다. 2i는 모델을 파는 것이 아니라 이렇게 측정된 패키징 작업, 즉 양자화 선택과 런타임 선택과 포팅 스코프 정의를 판다.

2 배경

2.1 왜 엣지에서 VLM을 돌리려 하는가

매장 운영 현장의 CCTV, 순찰 로봇의 카메라, 제조 현장의 검사 라인처럼 카메라가 이미 있는 곳에 비전-언어모델을 얹으려는 수요는 꾸준히 늘고 있다. 문제는 이런 현장 대부분이 데이터센터 GPU를 돌 전력, 공간, 네트워크 예산이 없다는 점이다. 영상을 클라우드로 실시간 전송해 추론하는 구조는 지연과 대역폭, 그리고 보안과 주권 요구사항(온프레임 처리) 앞에서 자주 막힌다. 그래서 질문은 단순해진다. SOTA 수준의 VLM을, 서버실이 아니라 매장 뒷방에 놓인 소비자용 컴퓨터 한 대에서 돌릴 수 있는가.

microsoft/Mage-VL은 이 질문에 답하기 좋은 대상이다. 이미지-텍스트 이해와 비디오, 스트리밍 이해를 함께 지원하는 4B 급 모델이면서, **Apache-2.0** 라이선스로 공개되어 있다. 이 라이선스가 이 논문의 존재 이유 중 하나다. 연구용 데모가 아니라 상업적으로 패키징해 고객사에 배치할 수 있는 모델이라는 뜻이고, 그래서 “이걸 실제로 소비자용 하드웨어에 올리면 무엇을 얻고 무엇을 내주는가”라는 질문이 사업적으로도 의미를 가진다.

2.2 모델 구조

Mage-VL은 두 부분으로 이루어진다. 시각 인코더는 자체 아키텍처인 Mage-ViT(4B 스케일)이고, 언어 백본은 Qwen3-4B-Instruct-2507이다. 우리가 실제로 로드해 확인한 파라미터 수는 **4.742B**다(공개 카드의 “4B” 표기와 합치한다). 모델은 여기에 두 가지를 더 얹는다. 하나는 “StreamMind”라는 이벤트 게이트로, 스트리밍 입력에서 능동적으로 주목할 순간을 트리거하기 위한 보조 가중치(1.07GB)다. 다른 하나는 DCVC-RT 신경 비디오 코덱으로, 비디오·스트리밍 경로의 프레임 처리를 가속하기 위해 C++/CUDA 확장으로 구현되어 있다.

이 구조에서 이번 갱신을 가능하게 만든 사실 하나가 나온다. 이미지 한 장, 혹은 비디오의 개별 프레임을 이해하는 경로는 이 신경 코덱을 아예 거치지 않는다. 그리고 자기회귀 디코딩(오토리그레시브 생성) 단계에서 비전 인코더는 더 이상 관여하지 않는다. 첫 토큰이 나올 때까지의 프리필 단계에서 이미지가 비전 토큰으로 한 번 변환되어 KV 캐시에 얹히면, 그 이후 토큰을 하나씩 만들어내는 디코드 루프는 순수하게 언어모델 트랜스포머 레이어를 반복 통과하는 연산이다. 다시 말해 디코드 처리량은 언어모델 바운드(LLM-bound) 연산이고, 비전 인코더의 구조나 크기와는 무관하게 언어 백본의 가중치 크기와 커널 효율이 지배한다. 이 사실이 이번 갱신의 실험 설계를 정당화한다. 백본만 격리해 다른 런타임에서 측정해도 디코드 속도의 지배 요인을 정확히 짚어낼 수 있다는 뜻이다.

2.3 저자 보고 수치 (참고, 우리 측정 아님)

Mage-VL 저자들은 8×B200 데이터센터 노드에서 벤치마크한 수치를 보고한다. 시각 토큰을 75% 이상 줄이고, 균일 프레임 샘플링 대비 최대 3.5배의 실제 체감(wall-clock) 속도 향상을 얻으며, 676개 토큰 기준으로 Food-101에서 96.1% 이상, ImageNet에서 86.3% 이상의 정확도를, SoccerNet 스트리밍 과제에서 F1 16.35, ROC-AUC 83.14를, OVO-Bench에서 64.00을 기록했다고 밝힌다. 이 수치들은 전부 **저자 보고**이며, 이 논문의 어떤 실측치와도 직접 비교하지 않는다. 우리가 이 논문에서 다루는 것은 완전히 다른 질문, 즉 “이 모델을 데이터센터가 아니라 소비자용 Mac에 올리면 무엇이 벌어지는가” 이다.

3 방법

3.1 하드웨어와 스택

측정 환경은 이전 측정과 동일하게 Apple M4 Pro, 통합 메모리 48GB, macOS, arm64 아키텍처의 일반 소비자용 컴퓨터다. 데이터센터 GPU가 아니라는 점을 다시 강조해 둔다. 두 개의 소프트웨어 스택을 같은 하드웨어에서 나란히 돌렸다. 첫째는 transformers 5.12 + torch 2.12, Apple Metal을 쓰는 MPS 디바이스로 Mage-VL 전체를 로드하는 경로다. 둘째는 mlx-lm으로 Qwen3-4B-Instruct-2507 언어 백본만 로드하는 경로다. 하드웨어를 고정하고 런타임과 커널 경로만 바꾼 것이 이번 실험의 핵심 통제 변수다.

3.2 양자화 방법 (두 경로)

quanto weight-only 경로는 이전 측정과 동일하다. optimum-quanto의 weight-only 경로를 언어모델 서브모듈 (language_model)에만 적용했다. 시각 인코더(Mage-ViT)는 세 정밀도 모두에서 bf16으로 유지했다. 이 경로는 가중치를 저비트로 저장하지만, 실제 행렬곱을 계산하는 순간에는 그 값을 다시 bf16/fp32로 즉석에서 역양자화한 뒤 계산한다.

mlx-lm 융합 커널 경로는 이번에 새로 추가한 경로다. Qwen3-4B-Instruct-2507 언어 백본을 MLX 포맷의 4비트 양자화 체크포인트로 변환해 mlx-lm으로 로드하고 디코드했다. MLX는 Apple Silicon의 통합 메모리 구조에 맞춰 설계된 배열 프레임워크로, 저비트 가중치를 역양자화 없이 직접 받아 계산하는 융합(fused) 커널을 갖고 있다. 이것이 quanto weight-only 경로와의 근본적인 차이다. 두 경로 모두 “가중치를 4비트로 저장한다”는 목표는 같지만, 하나는 계산 시점에 다시 풀어서 (dequantize) 계산하고 다른 하나는 저비트 표현 그대로 계산한다.

이와 별도로, 이전 측정에서 시도했던 커뮤니티 제작 Mage-VL INT4 체크포인트를 mlx-vlm으로 로드하는 시도도 그대로 결과에 포함한다. 이 시도는 mage_vl 아키텍처가 mlx-vlm 메인라인에 등록되어 있지 않아 로드 자체가 실패했지만, 온디스크 크기라는 유의미한 데이터 포인트를 남겼다.

3.3 테스트 입력

Mage-VL 전체 경로의 테스트 이미지는 이전 측정과 동일한 **합성 이미지**다. 실제 사진이 아니라 우리가 직접 만든 768×512 크기의 “역 전광판” 장면으로, 정답(ground truth)을 우리가 정확히 알고 있다는 것이 핵심이다. 제목은 “CENTRAL STATION - DEPARTURES”이고, 아래 세 줄에 각각 “14:05 / PLATFORM 2 / ON TIME”, “14:20 / PLATFORM 5 / DELAYED”, “14:35 / PLATFORM 1 / BOARDING”이 적혀 있으며, 기차 픽토그램과 빨간 STOP 표지판이 함께 그려져 있다. 실제 사진을 쓰지 않은 이유는 정답을 완전히 통제해 “모델이 실제로 화면을 읽었는가, 아니면 그럴듯하게 지어냈는가”를 명확히 판정하기 위해서다.

mlx-lm 백본 측정은 이미지를 입력하지 않는다. 언어 백본만 격리했으므로 텍스트 프롬프트에 대한 디코드 처리량을 측정했다. 이는 앞서 1.2절에서 밝힌 “디코드는 언어모델 바운드”라는 논거에 따른 설계이며, 이 격리가 왜 여전히 유효한 비교인지는 5절 한계 절에서 다시 명시적으로 짚는다.

3.4 측정 지표

측정한 지표는 다섯 가지다. **디바이스 메모리**는 각 런타임이 실제로 할당한 메모리량이다(Mage-VL 경로는 MPS 할당량, mlx-lm 경로는 MLX 피크 메모리). **디코드 처리량**은 초당 생성 토큰 수(tok/s)다. Mage-VL 경로는 128토큰 생성 구간의 평균이고, mlx-lm 백본 경로도 동일한 방식으로 측정했다. **프롬프트 처리량**(mlx-lm 전용 지표)은 프롬프트를 처리하는 속도(tok/s)로, Mage-VL 경로의 TTFT(첫 토큰까지 걸리는 시간, 초 단위)와 측정 단위가 다르다는 점을 표에서 명시한다. **품질**은 Mage-VL 전체 경로에서만 판정했다. 합성 이미지의 정답 요소(제목 문구, 3행의 시각·플랫폼·상태 정보, 기차 픽토그램, STOP 표지판)을 모델의 서술이 정확히 포함하는지를 육안으로 대조했다. mlx-lm 백본 단독 측정은 이미지 입력이 없으므로 품질 판정 대상이 아니다. **온디스크 크기**는 각 체크포인트의 저장 크기다.

4 결과

4.1 측정표

아래 표는 세 개의 Mage-VL 전체 경로 정밀도, mlx-vlm 시도, 그리고 이번에 추가한 mlx-lm 백본 융합 커널 경로까지 전부 정리한 것이다. mlx-lm 행은 Mage-VL 전체가 아니라 언어 백본만 격리한 측정이라는 점을 표 아래 각주로도 다시 밝힌다.

정밀도	런타임	대상	디바이스 메모리	디코드 tok/s	TTFT / 프롬프트 tok/s	품질	온디스크
bf16	transformers + MPS	Mage-VL 전체	9.49 GB	13.01	TTFT 2.94 s	정확함	9.8 GB
quanto INT8 (weight- only)	transformers + MPS	Mage-VL 전체	5.86 GB (-38%)	2.31	TTFT 1.77 s	정확함	메모리 내
quanto INT4 (weight- only)	transformers + MPS	Mage-VL 전체	4.15 GB (-56%)	0.47	TTFT 9.91 s	정확함 (표현만 재구성)	메모리 내
커뮤니티 INT4	mlx-vlm	Mage-VL 전체	로드 실패	측정 불가	측정 불가	측정 불가	2.9 GB
MLX 융합 INT4	mlx-lm	언어 백본만	2.46 GB	89.67	프롬프트 71.06 tok/s	해당 없음 (백본 단독)	해당 없음

각주: mlx-lm 행의 “디바이스 메모리”는 언어 백본 하나만 로드했을 때의 피크 메모리이며, Mage-ViT 비전 인코더를 포함한 Mage-VL 전체를 로드한 값이 아니다. “프롬프트 tok/s”는 mlx-lm이 보고하는 프롬프트 처리 속도이고, 다른 행의 TTFT(초 단위, 419토큰 프리필 후 첫 출력까지의 지연)와 측정 방법이 다르므로 두 값을 직접 나눴셈으로 환산하지 않는다.

같은 4비트 정밀도 두 행, quanto INT4(0.47 tok/s)와 MLX 융합 INT4 (89.67 tok/s)를 나란히 놓으면 비율은 $89.67 / 0.47 \approx 190.8$ 배다. 이 숫자가 이번 갱신에서 논문이 담는 핵심 격차다. 같은 비트폭, 같은 언어모델, 같은 하드웨어에서 런타임 (그리고 그 런타임이 가진 저비트 융합 커널의 유무)만 바꿨을 때 디코드 속도가 190배 가까이 벌어졌다.

bf16 Mage-VL 전체의 로드 시간은 약 12초였다. MLX 4비트 커뮤니티 mlx-vlm 체크포인트는 이전과 동일하게 커스텀 아키텍처 mage_vl이 등록되어 있지 않아(“Model type mage_vl not supported”) 로드 자체가 실패했다. mlx-lm 융합 경로는 이 문제를 겪지 않는데, 그 이유는 정확히 이 경로가 Mage-VL의 비전 인코더나 StreamMind 게이트를 전혀 포함하지 않고 표준 qwen3 아키텍처 백본만 로드하기 때문이다. 즉 mlx-lm이 성공한 것은 mlx-vlm의 실패를 해결한 것이 아니라, 실패 지점(비전 인코더 아키텍처 등록)을 아예 건너뛴 것이다. 이 구분을 5절에서 다시 명시한다.

4.2 그림: 메모리와 디코드 처리량, 정밀도와 런타임 둘 다 바꿔서

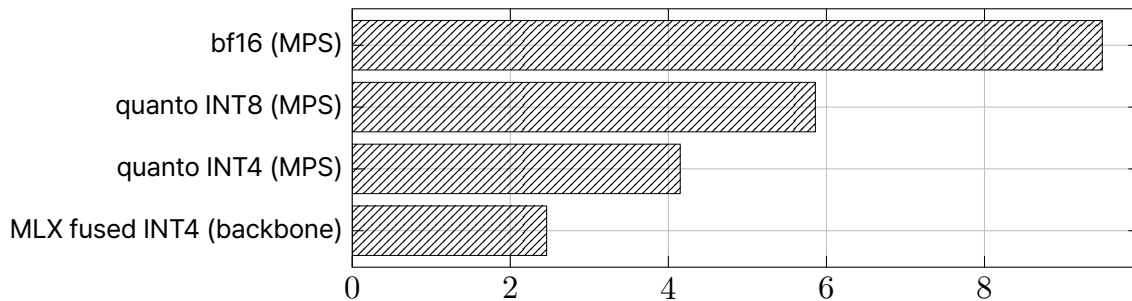


그림 1: 정밀도x런타임별 디바이스 메모리

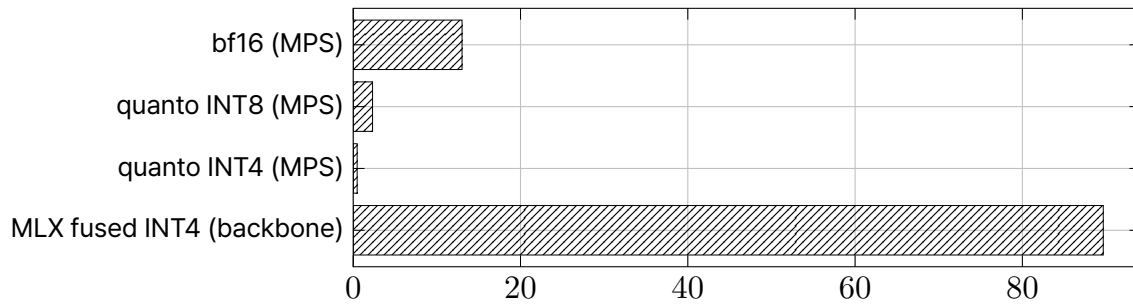


그림 2: 정밀도x런타임별 디코드 처리량

두 그림을 나란히 놓으면 이전 논문의 그림이 보여줬던 “메모리는 줄고 속도도 줄어드는” 어긋난 두 곡선에, 네 번째 막대가 추가되면서 이야기가 완성된다. 메모리 측에서는 MLX 융합 INT4가 가장 작다(2.46GB, 다만 백본 단독이라는 조건이 붙는다). 속도 측에서는 quanto 계열이 정밀도가 낮아질수록 계속 느려지다가, 똑같이 4비트인 MLX 융합 경로에서 갑자기 89.67 tok/s로 치솟는다. 두 그림 모두에서 네 번째 막대가 앞의 추세선을 정면으로 이탈하는 지점, 그것이 “런타임이 정밀도보다 더 결정적인 변수”라는 이 논문의 결론이 시각적으로 드러나는 자리다.

5 해석: 런타임이 레버다

이전 논문은 “메모리와 속도는 서로 다른 축”이라는 관찰에서 멈췄다. 양자화가 메모리를 줄이면서 속도까지 늦춘 이유를 커널 부재로 설명했지만, 그 설명이 맞다면 저비트 표현을 직접 계산하는 융합 커널이 있는 런타임에서는 같은 양자화가 속도까지 되찾아 줘야 한다는 예측이 함께 따라왔다. 이번 갱신은 정확히 그 예측을 검증하는 실험이었고, 결과는 예측과 방향과 크기 모두에서 일치했다.

같은 비트폭에서 커널이 전부를 바꾼다. optimum-quanto의 weight-only 경로는 가중치를 4비트로 저장하지만 Apple Metal(MPS)에는 이 표현을 직접 받아 계산하는 융합 저비트 행렬곱 커널이 없다. 그래서 매 순전파마다 역양자화라는 추가 연산이 붙고, 디코드에서는 이 비용이 토큰 하나마다 반복 청구되어 0.47 tok/s까지 떨어진다. mlx-lm은 같은 4비트 가중치를 MLX의 네이티브 저비트 커널로 직접 계산한다. 역양자화 단계가 아예 없으니 청구되는 추가 비용도 없다. 그 결과가 89.67 tok/s다. 메모리 대역폭 절감이라는 저비트 양자화의 본래 이점이, 이번에는 역양자화 비용에 잠식되지 않고 그대로 속도로 전환된 것이다.

메모리와 속도는 이제 같은 방향으로 움직인다. 이전 논문에서 지적한 “서로 다른 축”이라는 현상은 사실 축 자체의 성질이 아니라 quanto weight-only + MPS라는 특정 조합의 성질이었다. 같은 4비트 가중치를 MLX 융합 커널로 계산하면 메모리도 작고(2.46GB, 백본 단독 조건) 속도도 빠르다(89.67 tok/s). 축이 어긋난 것이 아니라, 잘못된 런타임이 한 축의 이득을 다른 축의 빛으로 전환시키고 있었던 것이다. 올바른 런타임을 쓰면 그 전환이 일어나지 않는다.

그래서 패키징 레시피는 “양자화”가 아니라 “양자화 + 런타임”이다. 이전 논문이 미래 과제로 남겨둔 가설, 즉 “옛지 양자화가 실제 속도 이득까지 내려면 저비트 표현을 직접 받아 계산하는 융합 커널을 가진 런타임이 필요하다”는 문장은 이제 가설이 아니라 190배라는 구체적인 숫자로 뒷받침된 결론이다. 배치 가능한 레시피는 단순히 “가중치를 4비트로 줄여라”가 아니라 “4비트로 줄이고, 그 4비트를 직접 계산하는 런타임 위에 올려라”다. 소비자용 실리콘 위에서 작고도 빠른 VLM을 원하는 고객에게 파는 것은 바로 이 두 번째 절반, 즉 런타임 선택과 그에 맞는 포팅 작업이다.

6 한계와 스코프

이 결과를 일반화하기 전에 분명히 밝혀야 할 경계가 여러 개 있다. 이번 갱신은 열린 질문 하나를 닫았지만 동시에 다음 작업을 더 구체적으로 정의했다.

190배는 언어 백본의 격리 측정이며, Mage-VL 전체의 엔드투엔드 수치가 아니다. 89.67 tok/s는 Qwen3-4B-Instruct-2507을 mlx-lm으로 단독 로드했을 때의 값이다. Mage-ViT 비전 인코더, StreamMind 이벤트 게이트, DCVC-RT 신경 코덱은 이 측정에 전혀 포함되지 않았다. 1.2절에서 밝혔듯 디코드는 언어모델 바운드 연산이므로 이 격리가 디코드 속도를 지배하는 요인을 정확히 짚어내지만, “Mage-VL 전체가 MLX에서 이 속도로 돈다”는 문장은 이 논문이 실측한 범위를 넘어선다. 정확한 진술은 “Mage-VL의 디코드 단계를 지배하는 언어 백본이, 융합 커널 런타임에서 같은 정밀도 대비 약 190배 빠르다”이다.

Mage-VL 전체를 융합 커널 런타임으로 옮기려면 아키텍처 포팅이 필요하다. 커스텀 아키텍처 `mage_v1`(Mage-ViT 비전 인코더, StreamMind 게이트를 포함)은 `mlx-vlm` 메인라인에 등록되어 있지 않다. 3.1절에서 확인했듯 이는 이번에도 실패했다. `mlx-lm` 경로가 성공한 이유는 이 문제를 해결한 것이 아니라, 표준 `qwen3` 아키텍처만 로드해 이 문제를 회피했기 때문이다. “작고도 빠른 Mage-VL 전체”를 얻으려면 `mage_v1`을 `mlx-vlm`(또는 동급의 MLX 비전-언어 프레임워크)에 포팅하는 구체적인 엔지니어링 작업이 여전히 남아 있다. 이것이 이 논문이 만들어내는, 실측으로 뒷받침된 판매 가능한 다음 작업이다.

메모리 비교는 대상이 다르다. quanto INT4의 4.15GB는 Mage-VL 전체 (비전 인코더 bf16 + 언어모델 INT4)의 메모리이고, MLX 융합 INT4의 2.46GB는 언어 백본만의 메모리다. 두 값을 비교하면 방향은 일치한다 (41% 더 작음)는 것도 흥미로운 신호이지만, 완전한 동급 비교는 아니다. 비전 인코더까지 포함한 전체 MLX 파이프라인의 메모리는 포팅이 끝나기 전까지는 측정할 수 없다.

스트리밍 비디오 경로는 여전히 스코프 밖이다. DCVC-RT 신경 코덱의 빠른 경로는 CUDA .cu 확장에 묶여 있고, Apple Metal에서는 그대로 실행되지 않는다. 이번 갱신도 이 경계를 바꾸지 못했다. 이미지·프레임 단위 이해는 이 코덱을 거치지 않으므로 이 논문의 측정과 무관하게 Metal에서 온전히 동작한다.

정확도 평가는 일반화되지 않는다. 품질 판정은 우리가 직접 만든 합성 이미지 한 장, 단일 프롬프트에 대한 정성적 대조다. 자동화된 스코어링도 아니고, 다양한 이미지·프롬프트에 대한 통계적 정확도도 아니다. `mlx-lm` 백본 단독 측정은 이미지를 입력하지 않으므로 품질 판정 자체가 없다.

저자 보고 수치는 우리 실측과 다른 조건이다. 1.3절에 인용한 시각 토큰 절감률, 속도 향상, Food-101, ImageNet, SoccerNet, OVO-Bench 수치는 8×B200 데이터센터 노드에서 저자들이 측정한 것이며, 우리의 소비자용 Mac 실측과는 하드웨어, 데이터셋, 평가 방법이 전혀 다르다. 두 수치 집합을 직접 비교하거나 하나를 다른 하나의 증거로 쓰지 않는다.

7 데이터와 재현

- **모델:** `microsoft/Mage-VL` (Hugging Face). 라이선스 Apache-2.0. Mage-ViT 비전 인코더 + Qwen3-4B-Instruct-2507 언어 백본. 로드된 파라미터 수 실측 4.742B. StreamMind 이벤트 게이트 가중치 1.07GB. DCVC-RT 신경 비디오 코덱 (C++/CUDA 확장 포함). 언어 백본 단독 참조는 Qwen/Qwen3-4B-Instruct-2507.
- **하드웨어:** Apple M4 Pro, 통합 메모리 48GB, macOS, arm64. 모든 경로 (transformers+MPS, `mlx-lm`)를 같은 하드웨어에서 측정.
- **소프트웨어 (Mage-VL 전체 경로):** `transformers` 5.12, `torch` 2.12, 디바이스 MPS. 양자화는 `optimum-quanto(weight-only, language_model)` 서브모듈에만 적용, 시각 인코더는 bf16 유지). MLX 비전-언어 시도는 `mlx-vlm`을 사용했다.
- **소프트웨어 (언어 백본 융합 커널 경로, 신규):** `mlx-lm`으로 Qwen3-4B-Instruct-2507을 4비트 MLX 포맷으로 로드해 디코드. 이 경로는 비전 인코더, StreamMind 게이트, DCVC-RT 코덱을 포함하지 않는다.
- **테스트 입력 (Mage-VL 전체 경로):** 합성 (synthetic) 768×512 “역 전광판” 이미지. 정답은 제목 “CENTRAL STATION - DEPARTURES”, 3행 (“14:05/PLATFORM 2/ON TIME”, “14:20/PLATFORM 5/DELAYED”, “14:35/PLATFORM 1/BOARDING”), 기차 픽토그램, 빨간 STOP 표지판으로 우리가 직접 정의했다. 실사진이 아니다. 언어 백본 단독 측정은 이미지 입력이 없다.
- **프롬프트/생성 설정 (Mage-VL 전체 경로):** “Describe this image in detail...” 형태의 상세 서술 요청. 프롬프트 길이 (비전 토큰 포함) 419토큰, 고정 생성 길이 128토큰.
- **측정 방법:** 디바이스 메모리는 각 런타임의 실제 할당량 (MPS 할당 또는 MLX 피크 메모리). 디코드 tok/s는 128토큰 생성 구간 평균 (두 경로 동일 방법). TTFT (Mage-VL 전체 경로)는 419토큰 프리필 완료 후 첫 출력 토큰까지의 지연. 프롬프트 tok/s (`mlx-lm` 경로)는 `mlx-lm`이 보고하는 프롬프트 처리 속도로, TTFT와 측정 방법이 다르다. 품질은 Mage-VL 전체 경로에서만, 정답 요소와의 정성적 대조로 판정.
- **재현 시 주의:** MLX 4비트 Mage-VL 전체 경로는 `mlx-vlm` 메인라인에 `mage_v1` 아키텍처가 등록되어 있지 않아 그대로 로드되지 않는다 (“Model type `mage_v1` not supported”). `mlx-lm` 백본 경로는 표준 `qwen3` 아키텍처만 사용하므로 이 문제 없이 재현 가능하다. Mage-VL 전체를 융합 커널 런타임에서 재현하려면 `mage_v1` 아키텍처 포팅이 선행되어야 한다.

8 참고문헌

1. Hugging Face 모델 카드: `microsoft/Mage-VL` (Apache-2.0), huggingface.co/microsoft/Mage-VL
2. Qwen/Qwen3-4B-Instruct-2507 모델 카드, huggingface.co/Qwen
3. Hugging Face transformers 라이브러리 문서, huggingface.co/docs/transformers

4. optimum-quanto (weight-only 양자화), github.com/huggingface/optimum-quanto
5. mlx-vlm (Apple MLX 비전-언어모델 런타임), github.com/Blaizzy/mlx-vlm
6. mlx-lm (Apple MLX 언어모델 런타임, 융합 저비트 커널), github.com/ml-explore/mlx-lm
7. Apache License 2.0 전문, apache.org/licenses/LICENSE-2.0